

GLA UNIVERSITY
Department of Computer Science & Engineering

PROJECT SYNOPSIS CUM REPORT

FraudShield – Real-Time Credit Card Fraud Detection Gateway

Name of Student	<i>Ayush Sharma</i>
University Roll No.	<i>22515500020</i>
Branch	<i>B.Tech</i>
Semester	<i>3rd Semester</i>
Course	<i>Computer Science and Engineering – Artificial Intelligence & Machine Learning</i>
Submitted To	<i>Mr. Vansh Agarwal, ML Faculty</i>
Submitted By	<i>Ayush Sharma</i>
Academic Year	<i>2026-2027</i>
Date of Submission	<i>[DD / MM / YYYY]</i>

A Synopsis cum Report Submitted in Partial Fulfilment of the ML Software Project

INTRODUCTION

Introduction to the Project

Credit card fraud is one of the most persistent challenges in modern digital commerce. Every time a card is swiped, tapped, or entered online, there is a small but real chance that the transaction is fraudulent — someone other than the legitimate cardholder attempting to make an unauthorized purchase. As more everyday spending moves online, where the card itself is never physically checked, this risk has only grown.

FraudShield is a real-time credit card fraud detection platform built to address this problem using machine learning. Rather than relying only on a fixed list of manual rules — for example, blocking a transaction simply because it crosses a certain amount — FraudShield learns patterns of genuine and fraudulent behaviour from historical transaction data, and uses that learning to judge new transactions as they happen.

The platform sits between a transaction being initiated and a decision being made about it. A transaction first passes through a quick rule-based check for obvious red flags, and then, for anything not immediately obvious, through a trained machine learning model that estimates how likely the transaction is to be fraudulent. The result is shown to the merchant instantly, and any transaction that looks suspicious is routed to a human fraud analyst for a final decision, rather than being silently blocked or silently allowed.

The project belongs to the domain of financial technology (FinTech), specifically the sub-area of applied machine learning for fraud and risk detection — a domain that sits at the intersection of banking, e-commerce, and data science. It is being built as the Machine Learning course project, and is designed around four kinds of users: the customer whose card is used, the merchant accepting the payment, the fraud analyst who reviews flagged cases, and the administrator who oversees the platform.

The overall goal of FraudShield is to demonstrate a working, end-to-end fraud detection system that is noticeably more accurate and adaptive than a simple rule-based approach, while remaining transparent enough that a human analyst always has the final say on genuinely uncertain cases.

PROBLEM STATEMENT

Statement of the Problem

Traditional fraud detection systems used by many merchants and payment processors rely heavily on static, rule-based logic: block transactions above a certain amount, block certain countries, block a card after a certain number of attempts in a short window. These rules are easy to understand and implement, but they suffer from two well-documented weaknesses.

First, they generate a high volume of false positives — legitimate transactions incorrectly flagged or declined because they happen to match a rule (a customer making an unusually large but genuine purchase, or travelling and using their card in a new country). Each false decline is a poor customer experience and, for the merchant, a lost sale. Second, static rules struggle to catch novel fraud patterns. Fraud tactics evolve constantly, and a rule written to catch last year's pattern often has little power against this year's.

This difficulty is compounded by the extreme rarity of fraud itself. In real-world, industry-representative datasets such as IEEE-CIS Fraud Detection — one of the two datasets used to train FraudShield's models — confirmed fraud cases make up only roughly 0.17% to 1.7% of all transactions. A system that simply predicted "not fraud" for every transaction would already be correct more than 98% of the time, which is exactly why raw accuracy is a misleading way to judge a fraud model, and why the small minority of true fraud cases are so easy for rigid, threshold-based rules to miss.

FraudShield's problem statement, therefore, is not simply "detect fraud," but specifically: how can a fraud detection system distinguish rare fraudulent transactions from an overwhelming majority of legitimate ones, in real time, without either over-flagging genuine customers or under-flagging real fraud — a balance that fixed, manually written rules are structurally unable to strike on their own.

PROBLEM STATEMENT

Scope & Areas of Impact

Although FraudShield is built and demonstrated around card transaction data, the underlying problem — distinguishing fraudulent from legitimate financial activity in real time — touches several industries and contexts beyond any single platform:

- **E-Commerce & Online Retail:** Card-not-present transactions, where the physical card is never checked, dominate online checkout and are the single largest surface for card fraud.
- **Banking & Financial Services:** Issuing banks and payment processors absorb the direct cost of confirmed fraud and carry the compliance burden of maintaining fraud-monitoring systems.
- **Travel & Hospitality:** High-value, often cross-border bookings (flights, hotels) are a frequent target, since large one-off purchases are harder to judge against a customer's normal spending pattern.
- **Digital Subscriptions & SaaS:** Recurring billing is exploited for small, repeated fraudulent charges that individually look unremarkable and can go unnoticed longer than a single large charge would.
- **Cross-Border / International Commerce:** Currency conversion, differing verification standards, and jurisdictional gaps between countries make international transactions both harder to verify and more attractive to fraudsters.

Within this project's own scope, FraudShield focuses specifically on card-not-present, e-commerce-style transactions, using publicly available transaction datasets rather than live financial data, since no real payment processor or banking data is accessible for a student project of this kind.

PROBLEM STATEMENT**Target Stakeholders & Segment Analysis**

The problem of card fraud does not affect a single, uniform group — it touches individual cardholders, the businesses that accept their payments, and the internal teams responsible for catching fraud before it causes damage. FraudShield's four user roles (Customer, Merchant, Fraud Analyst, Admin) map directly onto these real-world stakeholder segments:

Segment Name	Size / Scale	How They Are Impacted
Individual Cardholders (Customers)	Largest segment by headcount — every processed transaction involves an underlying cardholder	Bear the direct cost of fraud through unauthorized charges and card blocks, and when detection is too aggressive — the inconvenience of false declines on legitimate purchases
Small & Mid-Size E-Commerce Merchants	High in number but individually low-volume; typically lack an in-house fraud team	Absorb chargeback losses and processing penalties directly; a single false decline can mean a lost customer relationship
Large-Scale / High-Throughput Merchants	Fewer in number but process a disproportionately high share of total transaction volume	Need low-latency, high-accuracy screening at scale — even a small false-positive rate translates into large absolute revenue loss
Fraud Analysts (Internal Operations)	Small, specialized internal team per organization	Directly responsible for reviewing flagged transactions; overwhelmed by false positives from rule-only systems, under-resourced against novel fraud patterns
Platform Administrators	Very small — typically one or two per deployment	Responsible for system health, user/role management, and oversight of which trained model is currently live

Merchants are treated as the primary business-side segment, since they are both the direct financial victims of fraud (through chargebacks) and FraudShield's main dashboard users; fraud analysts and administrators are internal, operational stakeholders rather than external customers, but are equally central to the system functioning day to day.

PROPOSED SOLUTION

Minimum Viable Product (MVP) Definition

The Minimum Viable Product for FraudShield is the smallest version of the system that proves the core idea — that a trained machine learning model can screen transactions for fraud more usefully than a rule-only approach — end to end, for a single transaction flow.

Included in the MVP (must-have):

- Transaction Simulation Module — to generate realistic test transactions, since no live payment gateway is connected.
- Rule-Based Pre-Filter — the first-layer check for obvious fraud indicators.
- Fraud Detection Engine — real-time scoring using the model trained on the IEEE-CIS dataset (selected for production / live-traffic serving).
- A minimal Merchant view — showing each transaction's outcome (approved / flagged).
- A minimal Fraud Analyst view — a queue of flagged transactions with a confirm/override action.
- Basic notification when a transaction is flagged.

Not included in the MVP (Future Scope):

- The full Admin Panel (user/role management beyond a few seeded accounts).
- The Model Performance Analytics dashboard comparing all four trained models (2 algorithms × 2 datasets).
- Any active use of the secondary, human-readable-column dataset (Credit Card Fraud Detection 2026) — retained for future explainability work, not required for core detection.
- Streaming ingestion (Kafka), deep learning models, SHAP explainability, continuous/online learning, and any sandboxed real payment-gateway integration (Stripe/Plaid) — all explicitly Future Scope; the payment-gateway sandbox specifically is still under evaluation and has not been committed to the plan either way.

PROPOSED SOLUTION

Product Overview - Concept & Value Proposition

FraudShield is a web-based fraud detection platform that sits in the path of a card transaction and answers one question in real time: how likely is this specific transaction to be fraudulent? It is not a payment processor and does not move money — it is a decision-support layer that could sit in front of a merchant's or platform's existing payment flow.

The core value proposition is straightforward: **fewer wrongly blocked genuine customers, and fewer real fraud cases slipping through**, compared to a fixed rule list. For the customer, this means fewer embarrassing declines on legitimate purchases. For the merchant, it means fewer chargebacks and less revenue lost to both fraud itself and to over-cautious blocking. For the fraud analyst, it means a shorter, better-prioritized queue of transactions that genuinely deserve a second look, instead of manually sifting through everything a rigid rule engine happens to flag.

Where a purely rule-based system can only say "this matches a rule" or "it doesn't," FraudShield's underlying model produces a probability — a graded sense of risk — trained on real transaction patterns, which is what allows it to adapt to fraud behaviour that no one explicitly wrote a rule for.

PROPOSED SOLUTION

Product Overview - Working Principle & Differentiators

How it works, end to end: a transaction — simulated for demonstration purposes, standing in for a real card-not-present purchase — first passes through a lightweight rule-based pre-filter that catches unambiguous red flags cheaply, without needing a model call. Anything that isn't an obvious rule violation is passed to the Fraud Detection Engine, where a trained Random Forest or SVM model (loaded in-process via joblib, not as a separate service) scores it and returns a fraud probability. Based on that score, the transaction is either approved and shown to the merchant, or flagged and routed to a fraud analyst's queue for a human decision. Every outcome, human or automated, feeds back into the Model Performance Analytics view, so the system's real-world accuracy stays visible rather than assumed.

What sets FraudShield apart:

- **Dual-dataset training strategy:** Models are trained separately on two different fraud datasets: IEEE-CIS Fraud Detection for production-grade detection accuracy, and a second, human-readable dataset kept specifically because its non-anonymized columns leave room for future explainability work that IEEE-CIS's anonymized columns cannot support.
- **Layered defence, not a single gate:** Cheap rule checks handle the obvious cases; the more expensive ML model is reserved for genuinely ambiguous transactions.
- **Correct handling of an extremely imbalanced problem:** Class-balancing (SMOTE) is applied only after the train/test split to avoid data leakage, and the system is evaluated on precision, recall, F1, and AUC-ROC rather than accuracy, which is meaningless at fraud's real-world rate of well under 2% of transactions.
- **A human always has the final word:** Flagged transactions go to a fraud analyst, not straight to an automatic block, keeping a person accountable for consequential decisions.

PROPOSED SOLUTION**Rationale & Market Need**

The gap FraudShield addresses is not "no fraud detection exists" — most merchants and processors already run some form of it — but that a large share of real-world fraud detection still leans on static, manually maintained rule sets, either alone or as the dominant layer in front of any learned model. These rule sets are cheap to build and easy to explain, which is exactly why they remain common, but they carry two costs that this project treats as its reason to exist: legitimate customers absorb the cost of over-flagging, and fraud teams absorb the cost of chasing patterns the rules were never written to catch.

Compared to that baseline, FraudShield's value is in replacing a binary, rule-matched decision with a graded, learned probability — one trained on labelled historical data rather than hand-written thresholds — while still keeping the rule layer for cases where a simple check is genuinely sufficient and a full model call would be wasted effort. The benefit a merchant or fraud analyst gets that they don't get from a rules-only system is a detector that improves with better data and retraining, rather than one that only improves when someone notices a gap and writes a new rule for it.

This project also has a deliberate educational rationale: it is structured to demonstrate — honestly, including its constraints — what applying machine learning to a real, extremely imbalanced classification problem actually involves, from correct SMOTE placement to choosing the right evaluation metrics, rather than presenting a simplified version of the problem.

PROPOSED SOLUTION

Target Audience & User Personas

FraudShield's target audience is not a single consumer group but the operational side of digital commerce: small and mid-size online merchants who cannot afford a dedicated fraud team, and the fraud analysts who work at payment processors, banks, or larger merchants reviewing flagged activity. Both groups are working professionals, generally comfortable with dashboards and business software, based anywhere card-not-present commerce happens — which, practically, is global.

Primary Persona — Fraud Analyst

- **Name:** Riya Sharma
- **Age:** 27
- **Occupation:** Fraud Analyst at a mid-size payment processing company
- **Need:** A prioritized, trustworthy queue of transactions that actually deserve manual review, with enough context (score, reason, transaction detail) to decide quickly.
- **Pain Point:** Currently wades through a long list of rule-triggered alerts, most of which turn out to be legitimate customers, while genuinely unusual fraud patterns sometimes slip through unflagged.

Secondary Persona — E-Commerce Merchant

- **Name:** Arjun Mehta
- **Age:** 34
- **Occupation:** Owner-operator of a small online electronics store
- **Need:** To accept genuine payments smoothly while minimizing chargeback losses, without hiring a fraud specialist.
- **Pain Point:** Loses sales to false declines on real customers and absorbs losses from fraud that his current rule-based checkout flags too late, or not at all.

The **customer** (cardholder) and **platform administrator** are secondary, behind-the-scenes stakeholders rather than primary personas — the customer experiences FraudShield indirectly, through whether their genuine transaction goes through smoothly, and the administrator's needs are operational (system health, user management) rather than fraud-decision-making.

PROPOSED SOLUTION

Product Walkthrough - Structure & Scope

The walkthrough that follows demonstrates FraudShield through its four user roles, in the order a transaction naturally moves through the system: from submission, through automated screening, to human review and reporting. It is structured as eleven screens:

1. **Login & Role-Based Access:** Authentication and routing into the correct role-specific view (Customer / Merchant / Fraud Analyst / Admin).
2. **Transaction Simulation Module:** Generating or submitting a test transaction in place of a live payment gateway.
3. **Rule-Based Pre-Filter in Action:** The first-layer check and its immediate pass/fail outcome.
4. **Fraud Detection Engine — Scoring Result:** The ML model's fraud probability and decision for a given transaction.
5. **Merchant Dashboard — Transaction Overview:** A merchant's view of recent transactions and their outcomes.
6. **Merchant Dashboard — Transaction Detail:** A single transaction's fraud score breakdown.
7. **Fraud Analyst Dashboard — Flagged Queue:** The prioritized list of transactions awaiting human review.
8. **Fraud Analyst Dashboard — Case Review:** Reviewing a specific flagged transaction and recording a decision.
9. **Admin Panel — User & Role Management:** Provisioning and managing the four account types.
10. **Model Performance Analytics:** Precision, recall, F1, and AUC-ROC tracked across the system's models.
11. **Notification Centre:** Alerts delivered to merchants and customers when a transaction is flagged.

Screenshots for each of these eleven screens are presented individually on the following pages, once the corresponding build milestone is complete.

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 1

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 2

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 3

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 4

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 5

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 6

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 7

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 8

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 9

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 10

PROPOSED SOLUTION - PRODUCT WALKTHROUGH

Product Walkthrough - Screen 11

PROPOSED SOLUTION

Key Functionalities - Core Features

PROPOSED SOLUTION

Key Functionalities - Extended Features

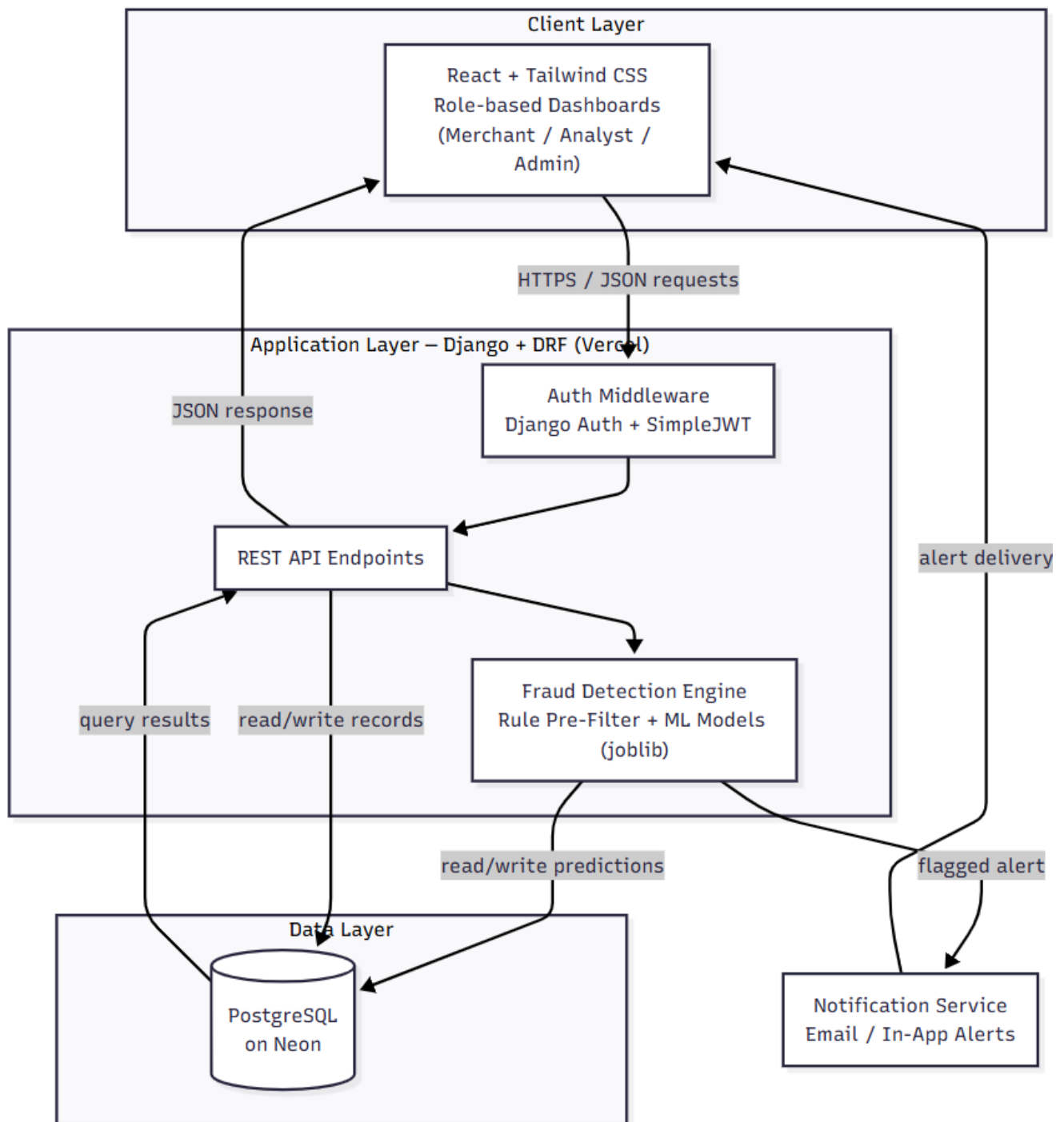
PROPOSED SOLUTION

Key Functionalities - Future Roadmap

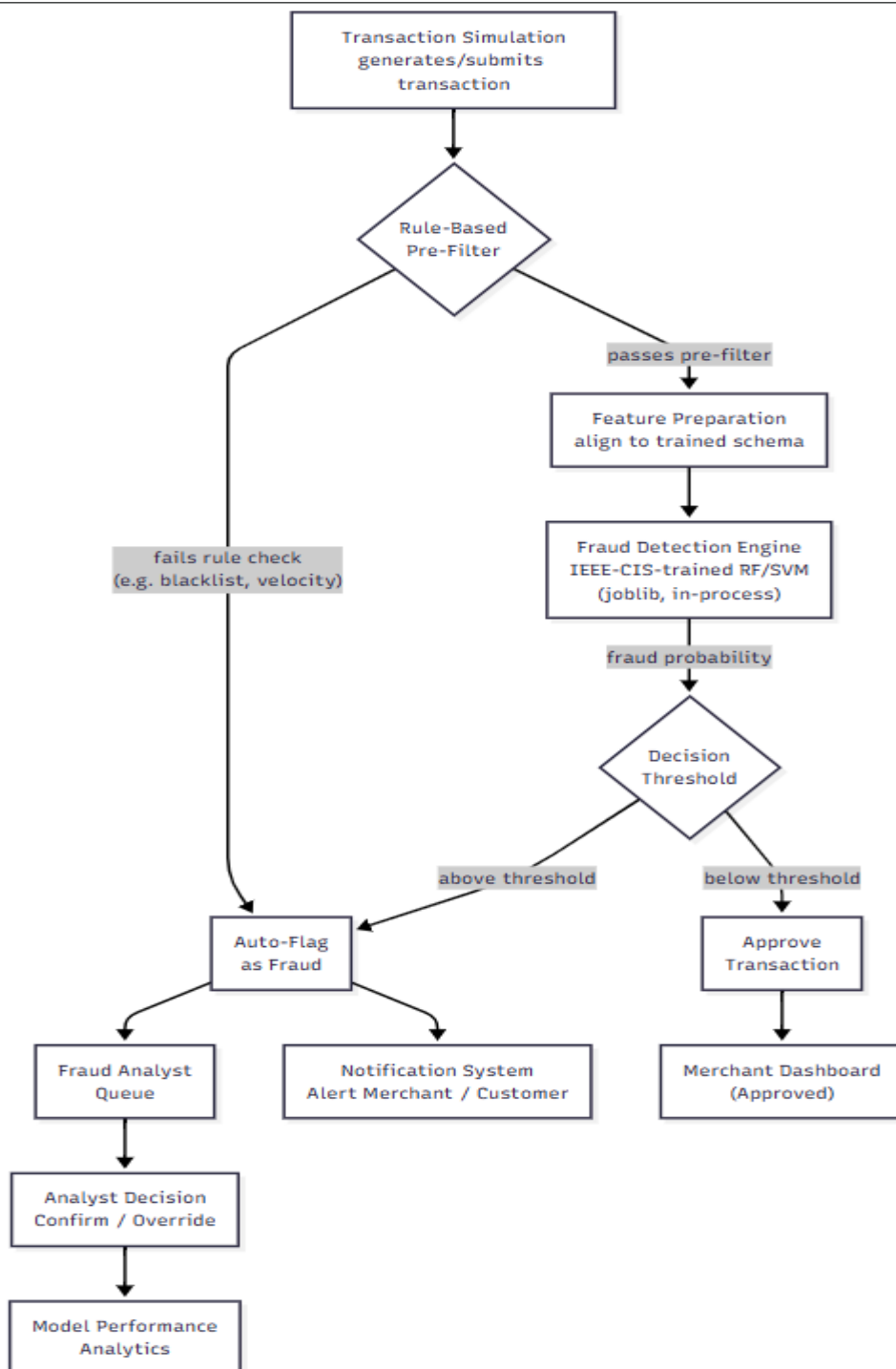
TECHNICAL ARCHITECTURE & DESIGN**Technology Stack Overview**

Layer	Technology Used	Purpose / Reason
Frontend	React.js + Tailwind CSS	Responsive, role-based dashboards (Merchant, Fraud Analyst, Admin) with fast utility-first styling
Backend / API	Django + Django REST Framework (DRF)	Serving the REST API, handling business logic, and orchestrating the rule pre-filter and ML scoring steps
Authentication	Django Auth + djangorestframework-simplejwt	Secure, token-based (JWT) authentication with role-based access control across four user types
Database	PostgreSQL	Relational integrity for interlinked transaction, prediction, and user data; native Django ORM support
ML / Data Science	scikit-learn, pandas, imbalanced-learn	Training and evaluating the Random Forest and SVM models; preprocessing and SMOTE-based class balancing
Model Serving	joblib	Loading trained models in-process within the Django application, avoiding a separate inference service
Data Visualization	Recharts	Rendering the Model Performance Analytics dashboard (precision, recall, F1, AUC-ROC trends)
Deployment / Hosting	Vercel (frontend + backend) + Neon (serverless PostgreSQL)	Unified deployment pipeline for the full stack, paired with a serverless, pooled-connection Postgres instance

Because trained models are loaded via joblib at process start, cold starts add latency on Vercel's serverless functions, and the scikit-learn version used to train each model must match the deployed environment exactly to avoid unpickling errors.

TECHNICAL ARCHITECTURE & DESIGN**System Architecture Diagram (Part 1 of 2)**

This is the system's overall component view: a React client talks to a Django/DRF backend over authenticated HTTPS, the backend runs the rule-filter-plus-ML fraud engine in-process and reads/writes a single PostgreSQL database on Neon, and a lightweight notification service pushes alerts back out to the client on flagged transactions.

TECHNICAL ARCHITECTURE & DESIGN**System Architecture Diagram (Part 2 of 2)**

This diagram zooms into the Fraud Detection Engine sub-system from Part 1, showing exactly how a single transaction moves from simulation through the rule pre-filter and the IEEE-CIS-trained model to a final decision. Transactions that fail the rule pre-filter or exceed the model's probability threshold are auto-flagged and routed to a fraud analyst rather than blocked outright. Every analyst decision feeds back into the Model Performance Analytics module, closing the loop between real-world outcomes and measured model accuracy.

TECHNICAL ARCHITECTURE & DESIGN

Database Design & Schema Overview

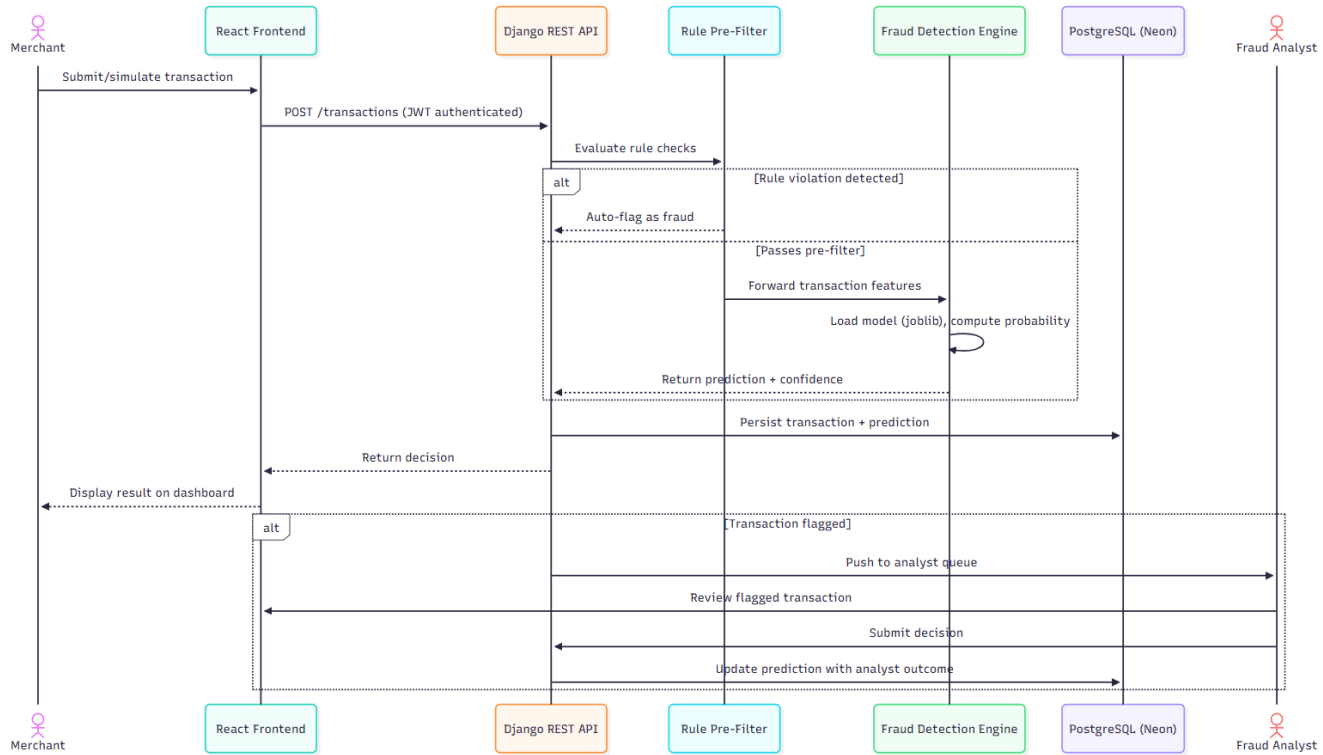
FraudShield uses **PostgreSQL**, a relational database, rather than a NoSQL/document store. The data itself is inherently relational — a transaction belongs to a merchant and (optionally) a customer, produces exactly one prediction, that prediction was produced by one specific model version, and may or may not be escalated to an analyst review — so the schema benefits from PostgreSQL's foreign-key constraints and join support to keep that structure consistent. PostgreSQL was also the natural choice given the project's existing Django + DRF backend (first-class ORM support) and Neon's serverless PostgreSQL hosting, already used for deployment.

Table	Description	Key Fields	Relationships
User	All platform accounts across the four roles	id, name, email, password_hash, role, created_at	One User (customer) -> many Transactions; one User (analyst) -> many AnalystReviews
MerchantProfile	Extended profile for accounts with merchant	id, user_id (FK), business_name, category	One-to-one with User; one MerchantProfile -> many Transactions
Transaction	A single card transaction (simulated, or live in future)	id, customer_id (FK), merchant_id (FK), amount, currency, timestamp, raw_features (JSON)	Many-to-one with User and MerchantProfile; one-to-one with FraudPrediction
MLModel	Metadata for each of the four trained models	id, algorithm (RF/SVM), dataset_source, version, is_production, trained_at, metrics (JSON)	One MLModel -> many FraudPredictions
FraudPrediction	The scoring result for one transaction	id, transaction_id (FK), model_id (FK), fraud_probability, predicted_label, rule_flagged, final_status	One-to-one with Transaction; many-to-one with MLModel; zero-or-one AnalystReview
AnalystReview	A human decision on a flagged prediction	id, prediction_id (FK), analyst_id (FK), decision, reviewed_at, notes	Many-to-one with FraudPrediction and with User (analyst)
Notification	An alert sent to a user	id, user_id (FK), transaction_id (FK, nullable), message, channel, sent_at, read_status	Many-to-one with User; many-to-one with Transaction

The `is_production` flag on **MLModel** is what determines which of the four trained models (2 algorithms × 2 datasets) actually serves live scoring requests — by design, this flag is only ever set on an IEEE-CIS-trained model, never on a model trained on the secondary dataset.

TECHNICAL ARCHITECTURE & DESIGN

UML Diagram - System Design Representation



This sequence diagram represents the end-to-end logic of a single transaction being screened, from submission through rule-based pre-filtering and ML scoring, to an optional human review step. A sequence diagram was chosen over a class or use-case diagram because FraudShield's most important design property is this real-time decision flow, not just its static data structure — which is already covered by the schema above.

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT

AI/ML Model Overview

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT

Data Collection & Preprocessing

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT

Model Selection & Architecture

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT

Model Training & Evaluation

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT

System Integration of the Model

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT

Performance Evaluation, Limitations & Future Enhancements

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING COMPONENT

Model Workflow / Pipeline Diagram

PROJECT EXECUTION TIMELINE

Execution Timeline - Overview (August to November)

Milestone	Target Month	Status/Notes

PROJECT EXECUTION TIMELINE

Month 1 - August

Task	Type (Build/Learn)	Time Required (Weeks)

PROJECT EXECUTION TIMELINE

Month 2 - September

Task	Type (Build/Learn)	Time Required (Weeks)

PROJECT EXECUTION TIMELINE

Month 3 - October

Task	Type (Build/Learn)	Time Required (Weeks)

PROJECT EXECUTION TIMELINE

Month 4 - November

Task	Type (Build/Learn)	Time Required (Weeks)

CITATIONS & DATA SOURCES

Citations - Datasets Used

CITATIONS & DATA SOURCES

Citations - APIs & Web Services

CITATIONS & DATA SOURCES

Citations - Research Papers & Journals

CITATIONS & DATA SOURCES

Citations - News Articles & Reports

CITATIONS & DATA SOURCES

Citations - Government / Open Data Portals

CITATIONS & DATA SOURCES

Citations - Images, Icons & Media Assets

CITATIONS & DATA SOURCES

Citations - Surveys & Market Studies

CITATIONS & DATA SOURCES

Citations - Other Data Sources

REFERENCES & LEARNING RESOURCES

References - Frontend Framework / Library

REFERENCES & LEARNING RESOURCES

References - Backend Framework

REFERENCES & LEARNING RESOURCES

References - Programming Languages

REFERENCES & LEARNING RESOURCES

References - Database Technology

REFERENCES & LEARNING RESOURCES

References - Cloud / Hosting Platform

REFERENCES & LEARNING RESOURCES

References - AI / ML Libraries & Frameworks

REFERENCES & LEARNING RESOURCES

References - Computer Vision / NLP Tools

REFERENCES & LEARNING RESOURCES

References - Version Control & DevOps Tools

REFERENCES & LEARNING RESOURCES

References - Testing Tools & Frameworks

REFERENCES & LEARNING RESOURCES

References - UI/UX Design Tools

REFERENCES & LEARNING RESOURCES

References - Security & Authentication

REFERENCES & LEARNING RESOURCES

References - Tutorials, Courses & Documentation
